UNIVERSIDAD NACIONAL DE INGENIERÍA

Facultad de Ciencias

Escuela Profesional de Ciencia de la Computación

- **Curso: Fundamentos de Programación**
- **Docente:** Américo Chulluncuy Reynoso

2026-1

Sesión 7:

Cadena de caracteres II

Contenido

1. Lectura y escritura de cadenas
2. Funciones para el manejo de cadenas
3. Funciones para caracteres
4. Funciones para convertir cadenas de caracteres

Lectura (input) y escritura(output) de cadenas

- **Lectura**: podemos ingresar una cadena de 2 formas:
  1. Usando **cin >>**
  2. Usando la función **`getline()`**

- **Escritura**: Una cadena puede ser mostrada utilizando **cout <<**

- ¿Por qué no necesitamos un bucle para mostrar una cadena?
  Porque **toda cadena tiene un delimitador**, el carácter nulo '\0'

- cin >> ingresa una cadena sin espacios (lee solo una palabra), debido a que el operador de extracción >> detiene la entrada cuando encuentra un espacio en blanco

Lectura de cadenas: getline()

- La función getline() puede ser utilizada **para ingresar texto de varias líneas**

- **Sintaxis**: cin.getline(**cadena**, **MAX**, **Caracter delimitador**), donde:
  - ✓**cadena** es el arreglo de caracteres. Es la variable en la que se almacena la lectura.
  - ✓**MAX** su valor determina el máximo número de caracteres permitidos durante la lectura.
  - ✓**Carácter delimitador**: Es el carácter que, al ser encontrado en el proceso de lectura, detiene la entrada. El carácter delimitador por default es '\n'.

- La función getline continua con la lectura de la cadena hasta que el número máximo de caracteres permitidos es introducido o hasta encontrar el carácter delimitador.

```cpp
#include <iostream>
using namespace std;

int main() {
    int edad;
    char nombre[100];

    cout << "Edad: ";
    cin >> edad;

    cin.ignore();  // limpia el '\n'

    cout << "Nombre completo: ";
    cin.getline(nombre, 100);

    cout << "Nombre: " << nombre << endl;
}
```

1. Funciones para el manejo de cadenas

Debemos incluir la librería:

```
#include <cstring>
```

Para más detalles de otras funciones en la librería, visitar: https://cplusplus.com/reference/cstring/

Función	Descripción
`strlen(s)`	Retorna la longitud (número de caracteres no nulos) de `c`.
`strcpy(s1, s2)`	Modifica `s1` reemplazando sus caracteres por copias de los caracteres de `s2`.
`strncpy(s1, s2, n)`	Modifica `s1` reemplazando sus caracteres por los `n` primeros caracteres de `s2`. Si `strlen(s2)` es menor que `n`, entonces `s1` es rellenado con 'ceros' hasta que un total de `n` hayan sido escritos en él.
`strcat(s1, s2)`	Modifica `s1` concatenando `s2` al final de `s1`.
`strncat(s1, s2, n)`	Modifica `s1` concatenando con los `n` primeros caracteres de `s2`. Si `strlen(s2)` es menor que `n`, entonces `s1` es rellenado con 'ceros' hasta que un total de `n` hayan sido concatenados al final de él.
`strcmp(s1, s2)`	Compara `s1` y `s2`, retornando un entero negativo, 0, o un entero positivo según `s1` es menor que, igual a, o mayor que `s2`.
`strstr(s1, s2)`	Busca en `s1` por la primera ocurrencia de `s2`, retornando un puntero al primer caracter de esta ocurrencia o `nullptr` si `s2` no es encontrado en `s1`.

Ejemplo 1

Dado la lista de nombres almacenados en un arreglo de cadenas,

{"Juan", "Maria", "Carlos", "Ana"};

utilice punteros a cadenas de caracteres y

1. Escriba una función que determine si un nombre está o no en el arreglo

2. Escriba una función que permita concatenar 2 nombres elegidos

3. Escriba una función que permita encontrar una subcadena

Ejemplo 2

Dada la siguiente cadena: “--Aqui tenemos, un ejemplo de cadena.”, escribir una función que elimine los caracteres - , y .
La salida debe ser: “Aqui tenemos un ejemplo de cadena”

2. Funciones para el manejo de caracteres

Agregar la libreria de cabecera: `#include <cctype>`

Función	Descripción
`islower(ch)`	Retorna `true` si `ch` es minúscula y `false` caso contrario.
`isupper(ch)`	Retorna `true` si `ch` es mayúscula y `false` caso contrario.
`tolower(ch)`	Retorna la minúscula equivalente a `ch` si `ch` es mayúscula.
`toupper(ch)`	Retorna la mayúscula equivalente a `ch` si `ch` es minúscula.

```
isalpha(ch);   // letra
isdigit(ch);   // dígito
isalnum(ch);   // letra o número
isspace(ch);   // espacio, tab, salto de línea, etc.
```

Para mayor detalle de funciones en la librería visitar: https://cplusplus.com/reference/cctype/

Ejemplo 3

Escribir una función que convierta una cadena de caracteres
de modo que las minúsculas se convierten en mayúsculas y
viceversa, mientras que los no alfabéticos se mantienen sin cambios.

Ejemplo

Entrada: "Veamos Su funCionamiento #@ 123! "
Salida: "vEAMOS sU FUNcIONAMIENTO #@ 123! "

3. Funciones para convertir cadenas

Agregar la libreria de cabecera: `#include <cstdlib>`

Función	Descripción
`atoi(s)`	Convierte `s` a un `int` (si es posible) y retorna este valor.
`atof(s)`	Convierte `s` a un `double` (si es posible) y retorna este valor.
`atol(s)`	Convierte `s` a un `long int` (si es posible) y retorna este valor.

Para mayor detalle de funciones en la librería visitar:
https://cplusplus.com/reference/cstdlib/

Ejemplo 4: uso de strtok()

Escribir un programa que reciba la siguiente cadena
"123, 45.67, -89, 1001, 23.45, 6789, 12.45"
y calcule la suma de todos los números enteros contenidos en ella

Más sobre cadenas

La clase **string** ofrece varias ventajas sobre las cadenas al estilo C:

Gran cuerpo de funciones miembro

Operadores sobrecargados para simplificar expresiones

Es necesario incluir el archivo de encabezado `#include <string>`

Para mayor detalle de la librería, visitar:
https://cplusplus.com/reference/string/string/

Ejemplo 5

Dada la cadena: `"Holaaaaa genioooo!!! holaaaa"`

Implemente un algoritmo que reduzca las letras consecutivas repetidas a un máximo de 2 ocurrencias.

Ejemplo:

```
Entrada:  "Holaaaaa genioooo!!! holaaaa"
 Salida:   "Holaa genioo!! holaa"
```

Ejercicios

1. Escribe una función que **recorra** una cadena de caracteres utilizando un puntero y vaya imprimiendo cada carácter en la pantalla.

2. Implementa una función que calcule la **longitud** de una cadena de caracteres utilizando punteros.

3. Crea una función que **compare** dos cadenas de caracteres utilizando punteros y devuelva un valor que indique si son iguales o no.

4. Escribe una función que **copie** una cadena de caracteres en otra utilizando punteros.

5. Implementa una función que **concatene** dos cadenas de caracteres utilizando punteros.

Ejercicios

6. Implementa una función que **elimine los caracteres duplicados** de una cadena de caracteres utilizando punteros.

7. Escribe una función que **cuente el número de palabras** en una cadena de caracteres utilizando punteros.

8. Implementa una función que **reemplace** todas las ocurrencias de una subcadena con otra subcadena en una cadena dada, utilizando punteros.

9. Implementa una función que **elimine los caracteres duplicados** de una cadena de caracteres utilizando punteros.

Ejercicios

10. Escribe una función que tome una cadena de entrada y un delimitador, y devuelva un arreglo de punteros a tokens.

11. Modifica la función anterior para que cuente el número de palabras en lugar de simplemente imprimir los tokens.

12. Crea una función que busque un token específico en una cadena y lo reemplace por otro token dado.

13. Escribe una función que tome una cadena de entrada con palabras separadas por un delimitador y las ordene alfabéticamente.